**NAME :KRISHNAPRIYA P**
**REG NO: 192511279**
**COURSE CODE:CSA0712**
**COURSE NAME: COMPUTER NETWORKS**
**CO5_Assessment Tool_1**
**Constrain Based Problem Solving**

# Q2. HTTP – Designing a Web Access Solution

## Scenario

The company has an internal web application named intranet.local. The application provides static resources such as HTML, CSS, JavaScript and image files, together with dynamic content generated by a backend server. The objective is to design an HTTP solution that gives efficient web access while controlling caching, reducing unnecessary network round trips, identifying the web server, and supporting a later migration from HTTP to HTTPS.

Because the application is intended for internal company use, the design also considers internal DNS, network access controls and firewall rules. The solution should allow employees inside the company network to access the application while preventing unnecessary exposure to the public Internet.

## a) HTTP Headers for Caching, No-Cache Dynamic Content, and Minimizing Round Trips

### 1. Caching static files for 24 hours

Static resources normally change less frequently than dynamic information. Therefore, HTML, CSS, JavaScript and image resources that are suitable for caching can be stored by the browser for 24 hours.

```
Cache-Control: public, max-age=86400
```

Here, public allows the response to be cached, while max-age=86400 specifies 86,400 seconds, equal to 24 hours. This reduces repeated downloads when users revisit the same page during the cache period.

```
HTTP/1.1 200 OK
Cache-Control: public, max-age=86400
```

The server may also provide an Expires header containing a date and time 24 hours in the future. However, Cache-Control is the preferred modern mechanism.

### 2. Preventing caching of dynamic content

Dynamic pages and data can contain information that changes frequently or may be specific to the current user. Such responses should not be stored in the browser cache.

```
Cache-Control: no-store
HTTP/1.1 200 OK
Cache-Control: no-store
```

For compatibility with older HTTP implementations, the server can additionally send Pragma: no-cache. The important policy is that dynamic content should be obtained freshly from the server.

## 3. Server Identification Header

Every response in the proposed design must identify the web server using the required Server header. The value specified for this scenario is IntraWeb/1.0.

```
Server: IntraWeb/1.0
```

This header provides a simple server identification mechanism and remains applicable after HTTPS is introduced because TLS protects the connection but does not remove normal HTTP response headers.

## 4. Minimizing Round Trips

Network performance can be improved by avoiding repeated connection establishment. The application should therefore use persistent HTTP connections, commonly called keep-alive connections. Multiple HTTP

requests can use the same TCP connection instead of creating a new connection for every file.

• Use one persistent TCP connection.

• Reuse the connection for multiple HTTP requests.

• Avoid separate TCP connections for each resource.

• Cache suitable static resources for 24 hours.

• Use HTTP/2 multiplexing when HTTPS and HTTP/2 are enabled.

If the application later uses HTTP/2 over HTTPS, multiplexing can improve efficiency because multiple resources can be transferred concurrently over one connection.

## b) Sequence of HTTP Requests and Responses

**Initial access to the dashboard**

Assume that a user accesses the following address for the first time:

```
http://intranet.local/dashboard
```

The dashboard contains one HTML file, one CSS file, one JavaScript file and one image file. Since this is the first visit, the browser does not yet have the required static resources in its cache.

**Step 1: DNS Resolution**

The browser first resolves intranet.local into an internal IP address. The request is handled by the company's internal DNS service, so the application can be resolved only within the appropriate company network.

```
intranet.local → 192.168.10.20
```

The IP address is an example; the actual address depends on the company's network configuration.

**Step 2: Establish TCP Connection**

After obtaining the server IP address, the browser establishes a TCP connection with the web server. For ordinary HTTP access, the default port is 80.

```
Client → Server
TCP connection → Port 80
```

**Step 3: Request the HTML Page**

The browser sends an HTTP GET request for the dashboard resource.

```
GET /dashboard HTTP/1.1
Host: intranet.local
Connection: keep-alive
```

The server returns a successful response. Because the dashboard is dynamic, the response is not stored in the browser cache.

```
HTTP/1.1 200 OK
Server: IntraWeb/1.0
Content-Type: text/html
Cache-Control: no-store
Connection: keep-alive
```

## Step 4: Browser Discovers Required Resources

After receiving the HTML document, the browser parses it and identifies the resources required to render the dashboard.

```
/dashboard
├── style.css
├── script.js
└── image.jpg
```

The browser then requests the CSS, JavaScript and image resources. Since the TCP connection is persistent, the browser can reuse the existing connection.

**CSS Request and Response**

```
GET /style.css HTTP/1.1
Host: intranet.local
Connection: keep-alive
HTTP/1.1 200 OK
Server: IntraWeb/1.0
Content-Type: text/css
Cache-Control: public, max-age=86400
```

**JavaScript Request and Response**

```
GET /script.js HTTP/1.1
Host: intranet.local
HTTP/1.1 200 OK
Server: IntraWeb/1.0
Content-Type: application/javascript
Cache-Control: public, max-age=86400
```

**Image Request and Response**

```
GET /image.jpg HTTP/1.1
Host: intranet.local
HTTP/1.1 200 OK
Server: IntraWeb/1.0
Content-Type: image/jpeg
Cache-Control: public, max-age=86400
```

## How the Complete First Visit Works

The complete sequence begins with DNS resolution and continues with TCP connection establishment. The browser then requests the dynamic dashboard, receives the HTML response, discovers the static dependencies and requests each required resource. The server identifies itself with the Server header and applies different cache policies to dynamic and static content.

• DNS maps intranet.local to the internal server address.

• TCP establishes communication with the web server.

• GET /dashboard retrieves the dynamic dashboard.

• The HTML identifies style.css, script.js and image.jpg.

• The browser requests the static resources using the persistent connection.

• Static resources are cached for 24 hours.

• Dynamic content is marked no-store.

## How the Design Minimizes Round Trips

The design reduces unnecessary network round trips through connection reuse and caching. On the first visit, the required resources must be transferred. On later visits, resources still within their 24-hour cache lifetime can be reused locally, so the browser does not need to download them again.

## c) Changes Required When HTTPS Is Enabled

If the company later enables secure access through HTTPS, the basic application structure can remain the same, but the transport security configuration must be changed. HTTPS uses TLS to protect communication between the browser and the web server. The internal nature of the application should also be retained.

### 1. Obtain and Install a TLS Certificate

The web server requires a valid TLS certificate for the hostname used by the application, such as intranet.local. Since the application is internal, the company can use an internal Certificate Authority if company-managed devices trust the internal CA.

### 2. Configure HTTPS

The web server must be configured to support HTTPS using TLS. HTTPS normally uses TCP port 443 rather

than HTTP port 80.

```
HTTPS → Port 443
```

The browser establishes a secure TLS session before exchanging application data. This protects HTTP requests and responses against interception and modification while they travel across the network.

**3. Keep the Same Caching Headers**

Moving from HTTP to HTTPS does not automatically change the caching policy. Static resources can continue to use the 24-hour cache policy, while dynamic responses can continue to use no-store.

```
Static content: Cache-Control: public, max-age=86400
Dynamic content: Cache-Control: no-store
```

**4. Keep the Server Header**

The application should continue to include the required server identification header in HTTPS responses.

```
Server: IntraWeb/1.0
```

**5. Maintain Internal-Only Access**

HTTPS should not mean that the application must become publicly accessible. Internal DNS, firewall rules, network access controls and internal routing can continue to restrict access to authorized company networks.

• Use internal DNS for intranet.local.

• Apply firewall rules to restrict access.

• Use network access controls where appropriate.

• Maintain internal routing.

• Avoid exposing the application directly to the public Internet.

**6. Redirect HTTP to HTTPS**

If port 80 remains enabled, the server can redirect users from HTTP to HTTPS.

```
HTTP/1.1 301 Moved Permanently
Location: https://intranet.local/dashboard
```

After the redirect, the browser accesses the dashboard through the encrypted HTTPS connection.

## 7. Use Persistent Connections and HTTP/2

HTTPS can also use persistent connections. When HTTP/2 is supported, the browser can multiplex multiple requests and responses over the same secure connection. This is useful for a dashboard containing several resources because CSS, JavaScript and image transfers can be handled efficiently without requiring a separate connection for each resource.

The combination of HTTPS, persistent connections and HTTP/2 therefore provides both secure communication and improved network efficiency. The original cache rules remain applicable to static and dynamic content.

## Final HTTPS Design

```
User Browser
      ↓
Internal DNS
      ↓
intranet.local
      ↓
Internal Web Server
      ↓
HTTPS / TLS — Port 443
      ↓
HTTP/2 + Persistent Connection
      ↓
Dashboard + CSS + JavaScript + Image
```

The final design keeps the application within the internal network while adding TLS protection. Static resources can remain cached for 24 hours, and dynamic content can remain non-cacheable. The Server header continues to identify the server, while HTTP/2 and persistent connections help reduce communication overhead.

## Comparison of HTTP and HTTPS Designs

| Feature | HTTP Design | HTTPS Design |
|---|---|---|
| Protocol | HTTP | HTTPS using TLS |
| Typical port | 80 | 443 |
| Static caching | 24 hours | 24 hours |
| Dynamic content | Cache-Control: no-store | Cache-Control: no-store |
| Server header | IntraWeb/1.0 | IntraWeb/1.0 |
| Connection | Persistent TCP | Persistent TLS |
| HTTP/2 | Not required | Can be enabled |
| Internal access | Internal DNS/firewall | Internal DNS/firewall + TLS |

## Conclusion

The proposed HTTP solution provides an efficient design for the internal intranet.local application. Static resources such as CSS, JavaScript and images are cached for 24 hours using Cache-Control: public, max-age=86400, while dynamic content is protected from browser storage using Cache-Control: no-store. The Server header identifies the web server as IntraWeb/1.0.

The request sequence begins with internal DNS resolution, followed by TCP connection establishment and the HTTP GET request for the dashboard. The browser then discovers and requests the CSS, JavaScript and image resources. Persistent connections reduce repeated connection setup, and cached static resources reduce network traffic during later visits.

When HTTPS is introduced, the server must use a valid TLS certificate, configure HTTPS on port 443 and retain the existing caching and server-identification requirements. Internal DNS, firewall rules and network access controls should continue to restrict access to the company network. Redirecting HTTP to HTTPS and enabling HTTP/2 can provide a more secure and efficient final design.